Proyecto Base de Datos: Gestión de Recolección de Aceitunas

Índice

1. Descripción de la temática del proyecto **3**
2. Resolución del diagrama MER extendido **3**
2.1. Entidades *4*
2.2. Relaciones *5*
3. La transformación a Modelo Relacional 5
4. Carga Masiva de Datos **6**
5. Consultas Multitabla SQL **13**
5.1 Consulta 1 – (4 tablas) *13*
5.2 Consulta 2 – GROUP BY *14*
5.3 Consulta 3 – Subconsulta *15*
5.4 Consulta 4 – Funciones de fecha *16*
5.5 Consulta 5 – Funciones de texto *17*
5.6 Consulta 6 – ISNULL / IFNULL *18*
5.7 Consulta 7 – Consulta compleja *19*
6. Vistas **20**
6.1 Vista 1 ..... *20*
6.2 Vista 2 ..... *21*
7. Scripts SQL del Proyecto **22**
8. Archivos en git **23**
9. Conclusión y valoración personal Parte 1 ..... **24**

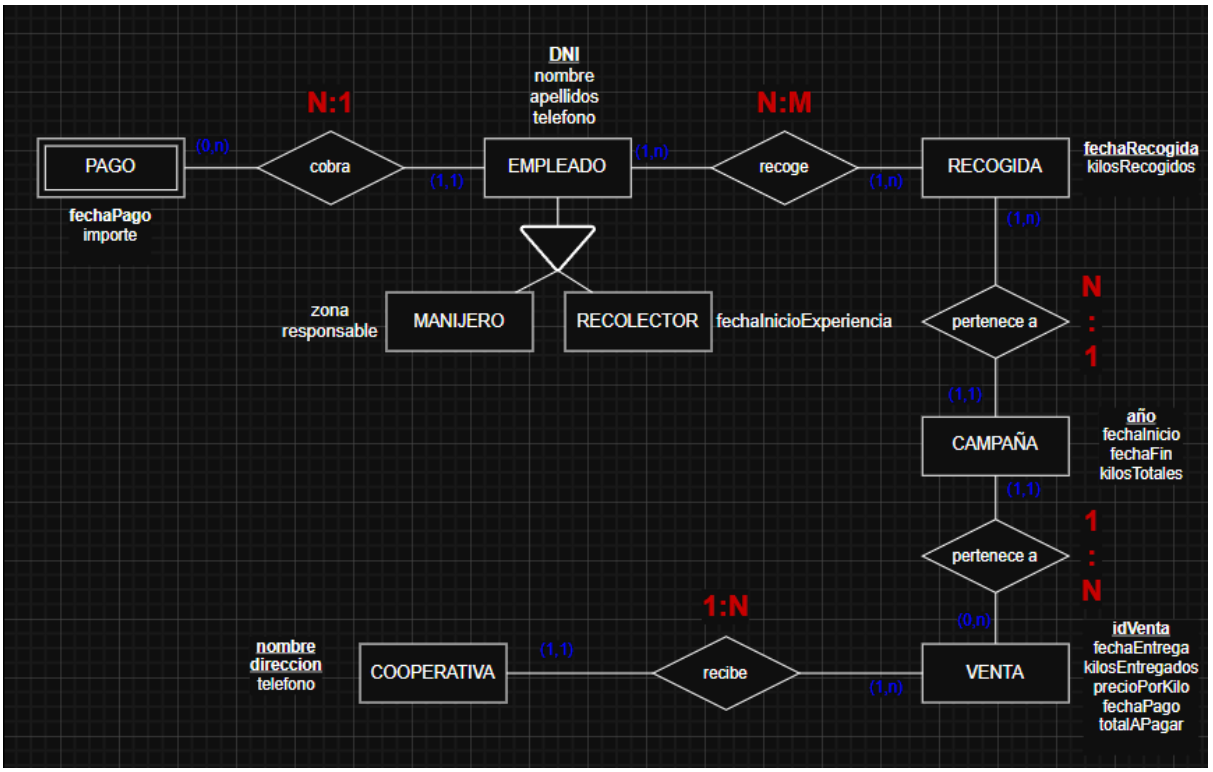
1. Descripción de la temática del proyecto.

He hecho este proyecto porque tengo una finca con olivos y quería crear una base de datos que sirviera para organizar la recogida de aceitunas.

El sistema permite registrar los empleados, las campañas, los pagos, las cooperativas y las ventas, para llevar un control más claro y automatizado.

Con este diseño se puede saber quién trabaja, cuánto se recoge y cuánto se paga o se vende, ayudando a gestionar mejor cada campaña.

2. Resolución del diagrama MER extendido



2.1. Entidades

Entidad EMPLEADO (Padre de la Herencia)

Guarda los datos personales del trabajador (**DNI, nombre, apellidos y teléfono**).

Un empleado puede ser de dos tipos y puede ser los 2 a la vez:

- **MANIJERO (Herencia)**, que tiene una zona a su cargo (Tiene que controlar a los recolectores y decir hoy se coje en tal zona y tales olivos y se para a tal hora).
- **RECOLECTOR (Herencia)**, que tiene un año para saber en qué año empezó a recoger aceitunas.

Entidad PAGO (Débil)
Registra los pagos realizados a los empleados, con su **fecha** e **importe**.

Entidad RECOGIDA
Guarda los datos de cada recogida (**fecha y kilos recogidos**).

Entidad CAMPAÑA
Representa cada temporada de trabajo (**año, fechas de inicio y fin, kilos totales** recogidos de cada campaña).

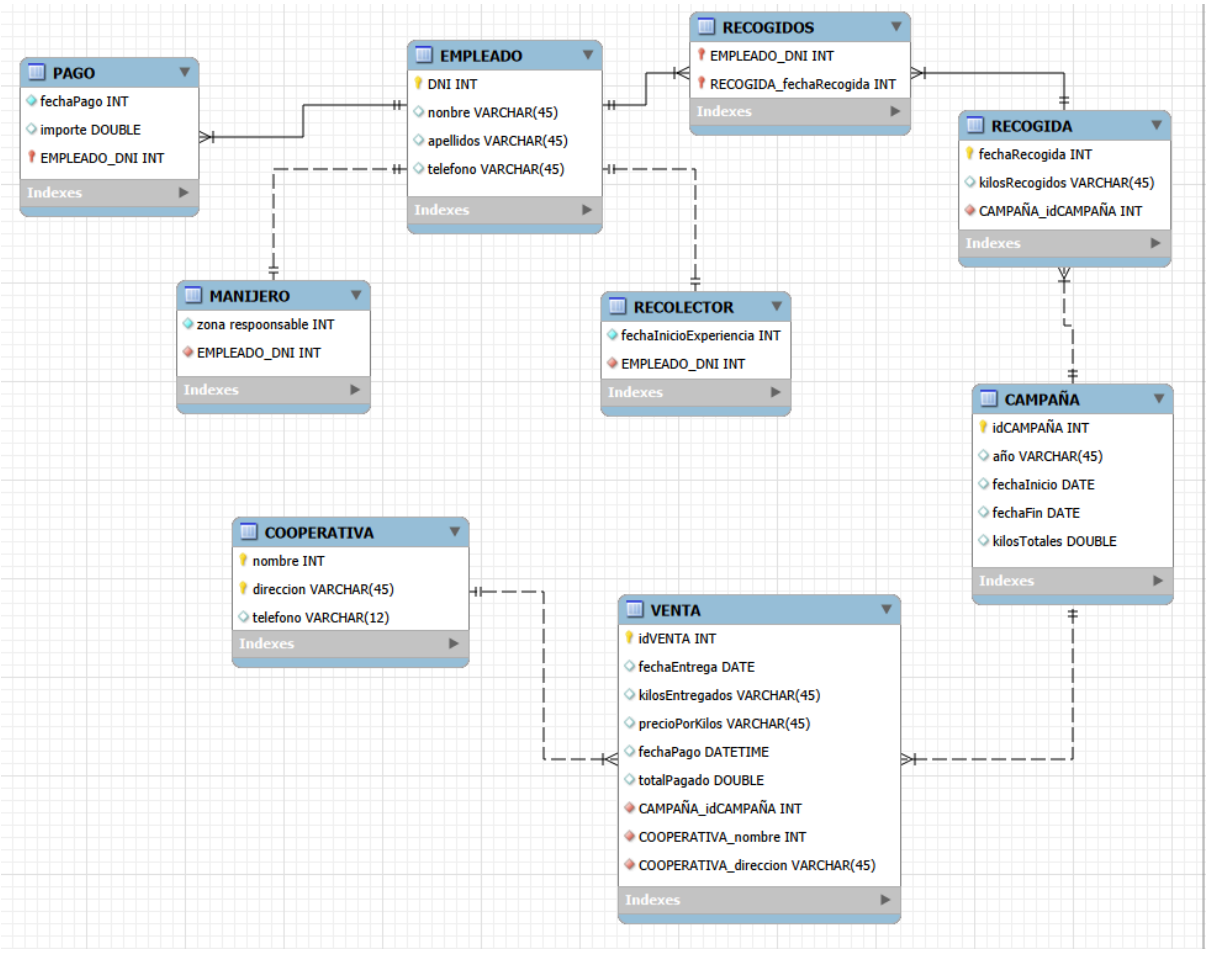
Entidad VENTA
Registra las ventas de aceitunas (**idVenta, fecha, kilos entregados y precio por kilo, fechaPago, totalAPagar**).

Entidad COOPERATIVA
Guarda los datos de las cooperativas a las que se entregan las aceitunas (**nombre, dirección y teléfono**).

2.2. Relaciones

- **PAGO – EMPLEADO (N:1):** Un empleado puede tener muchos pagos, pero un pago pertenece a un solo empleado.
- **EMPLEADO – RECOGIDA (N:M):** Ya que varios empleados pueden participar en distintas recogidas.
- **RECOGIDA – CAMPAÑA (N:1):** Una campaña incluye varias recogidas, y una recogida pertenece a una sola campaña.
- **CAMPAÑA – VENTA (1:N):** Cada campaña puede tener varias ventas, pero una venta pertenece a una sola campaña.
- **VENTA – COOPERATIVA (N:1):** Una cooperativa puede recibir muchas ventas, pero una venta es a solamente a una cooperativa.

3. La transformación a Modelo Relacional

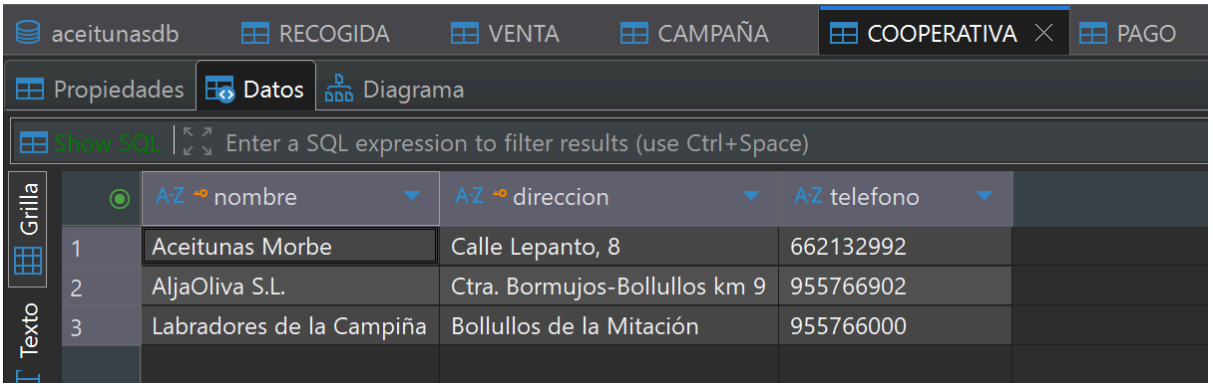


4. Carga Masiva de Datos

Para completar este apartado, he utilizado Gemini AI como apoyo para generar gran parte de los datos. Al haberle facilitado previamente la estructura de la base de datos, ha sido más sencillo obtener archivos CSV con información coherente, como nombres reales, DNI que encajan entre tablas y fechas lógicas para una campaña agrícola.

En el caso de la tabla COOPERATIVA, he optado por usar SQL por qué son 3 cooperativas, con estas líneas de código:

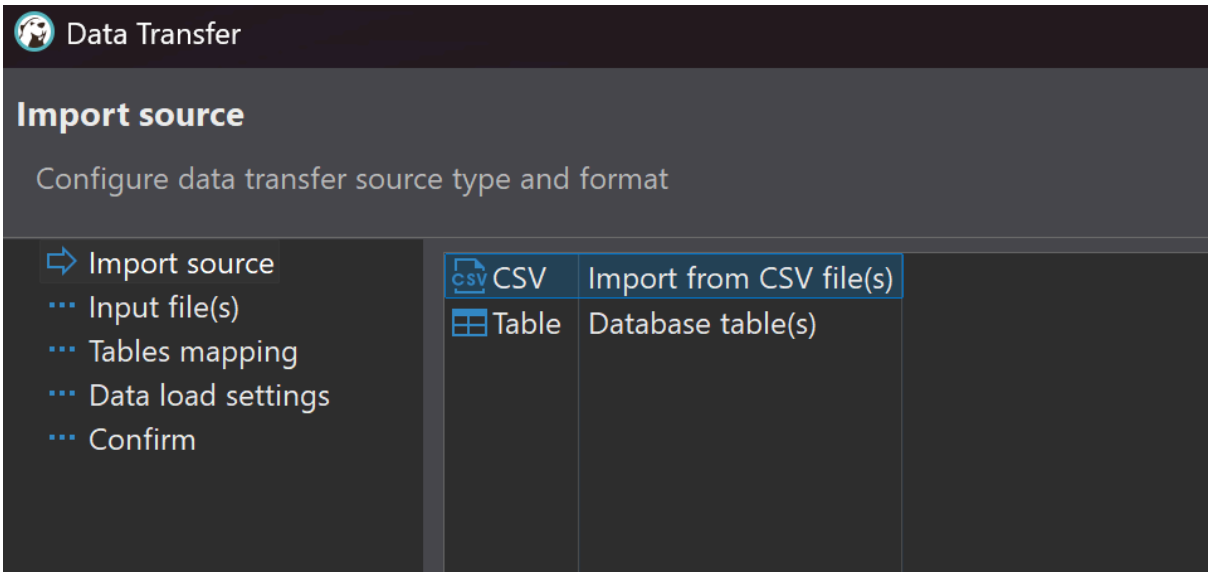
```
INSERT INTO `aceitunasdb`.`COOPERATIVA` (nombre, direccion, telefono) VALUES
('AljaOliva S.L.', 'Ctra. Bormujos-Bollullos km 9', 955766902),
('Aceitunas Morbe', 'Calle Lepanto, 8', 662132992),
('Labradores de la Campiña', 'Bollullos de la Mitación', 955766000);
```



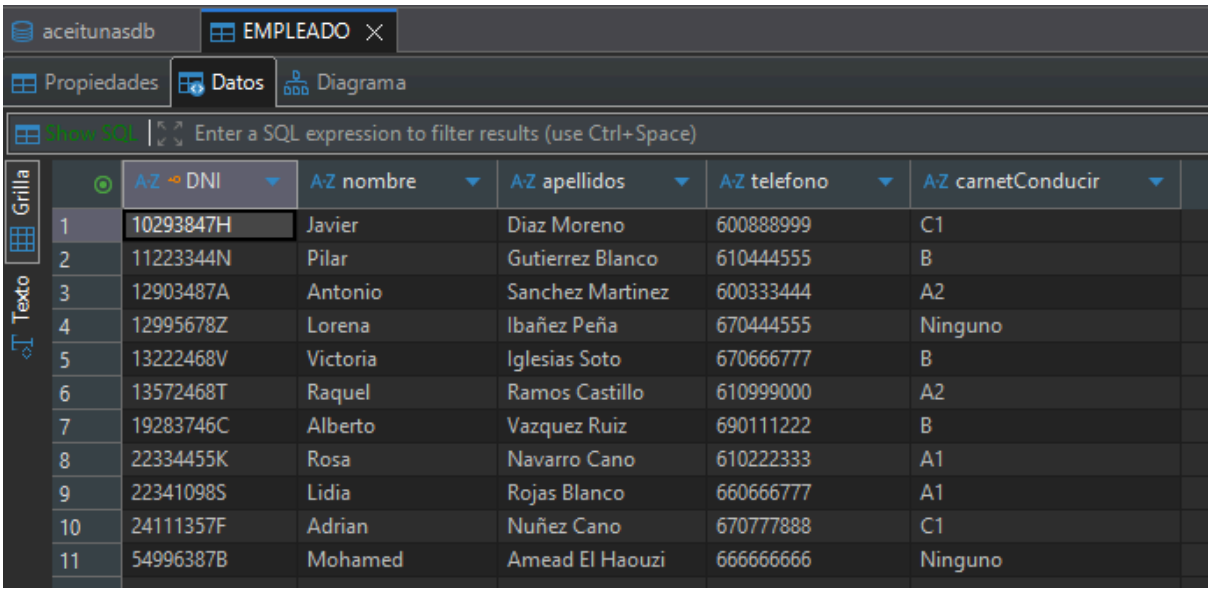
Finalmente, toda la información se ha metido en la base de datos de AWS usando el asistente de importación de DBeaver, revisando que se mantuvieran las claves foráneas.

Hago un paso a paso de como e importado la primera tabla y ya todos así:

1. Nos descargamos los CSV correspondientes.
2. Nos vamos al DBeaver a la tabla que queramos importarles los datos.
3. Click derecho a la tabla > Import Data > CSV:



4. Le damos a siguiente y seleccionamos el archivo desde nuestro explorador de archivos.
5. Le damos a todo “Siguiente” y por último a “Continuar”, y se meterán todos los datos:



	DNI	nombre	apellidos	telefono	carnetConducir
1	10293847H	Javier	Diaz Moreno	600888999	C1
2	11223344N	Pilar	Gutierrez Blanco	610444555	B
3	12903487A	Antonio	Sanchez Martinez	600333444	A2
4	12995678Z	Lorena	Ibañez Peña	670444555	Ninguno
5	13222468V	Victoria	Iglesias Soto	670666777	B
6	13572468T	Raquel	Ramos Castillo	610999000	A2
7	19283746C	Alberto	Vazquez Ruiz	690111222	B
8	22334455K	Rosa	Navarro Cano	610222333	A1
9	22341098S	Lidia	Rojas Blanco	660666777	A1
10	24111357F	Adrian	Nuñez Cano	670777888	C1
11	54996387B	Mohamed	Amead El Haouzi	666666666	Ninguno

Seguidamente hacemos la de Manijeros:

aceitunasdb | MANIJERO

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	zonaResponsable	EMPLEADO_DNI
1	Finca El Pintao - Sector Norte	10293847H
2	Finca El Pintao - Sector Sur	11223344N
3	Finca El Pintao - Sector Medio	54996387B

Y la de recolectores:

aceitunasdb | EMPLEADO | RECOLECTOR

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	fechaInicioExperiencia	EMPLEADO_DNI
1	1988	12903487A
2	2008	12995678Z
3	2003	13222468V
4	2002	13572468T
5	1998	19283746C
6	2005	22334455K
7	2001	22341098S
8	1995	24111357F

La de campaña:

aceitunasdb | EMPLEADO | RECOLECTOR | CAMPAÑA

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	idCampaña	año	fechaInicio	fechaFin	kilosTotales
1	1	2023	2023-07-10	2023-12-31	730.736
2	2	2024	2024-07-01	2024-12-20	1.111.112
3	3	2025	2025-07-01	2025-11-20	750.000

La de recogida que están todos los días 1 a 1 desde fecha inicio a fecha fin que se indica en campaña:

aceitunasdb | RECOGIDA

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla | Texto

	fechaRecogida	kilosRecogidos	CAMPAÑA_idCAMPAÑA
1	2023-07-10	3.970	1
2	2023-07-11	3.975	1
3	2023-07-12	3.965	1
4	2023-07-13	3.980	1
5	2023-07-14	3.972	1
6	2023-07-15	3.973	1
7	2023-07-16	3.980	1
8	2023-07-17	3.967	1
9	2023-07-18	3.975	1
10	2023-07-19	3.970	1
11	2023-07-20	3.975	1
12	2023-07-21	3.965	1
13	2023-07-22	3.980	1

(491 Filas)

Ahora con recogidos:

aceitunasdb | RECOGIDOS

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla | Texto

	EMPLEADO_DNI	RECOGIDA_fechaRecogida
1	10293847H	2023-07-10
2	11223344N	2023-07-10
3	12903487A	2023-07-10
4	12995678Z	2023-07-10
5	13222468V	2023-07-10
6	13572468T	2023-07-10
7	19283746C	2023-07-10
8	22334455K	2023-07-10
9	12903487A	2023-07-11
10	12995678Z	2023-07-11
11	13222468V	2023-07-11
12	12995678Z	2023-07-12
13	13222468V	2023-07-12
14	13572468T	2023-07-12
15	19283746C	2023-07-12
16	22334455K	2023-07-12
17	22341098S	2023-07-12
18	24111357F	2023-07-12

(3183 Filas)

Ahora la tabla pagos, he pagado 50€ al día y voy a pagar por cada mes, y como lo tenía todos los trabajadores han ido todos los días pues era más fácil de calcular, pero como he cambiado y he borrado muchísimos datos para que no sea todo simétrico e borrado, pero al ser muchos datos me lío mucho pero bueno lo que hay que saber es cargar por csv que sé y hacer consultas que las enseñaré más adelante.

*aceitunasdb | RECOGIDA | RECOGIDOS | *<AWS-AceitunasDB> Script-14 | CAMPAÑA | PAGO

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla | Texto

	EMPLEADO_DNI	fechaPago	importe
1	10293847H	2023-07-31	650
2	10293847H	2023-08-31	1.300
3	10293847H	2023-09-30	1.400
4	10293847H	2023-10-31	1.550
5	10293847H	2023-11-30	1.500
6	10293847H	2023-12-31	1.550
7	10293847H	2024-07-31	1.400
8	10293847H	2024-08-31	1.400
9	10293847H	2024-09-30	1.400
10	10293847H	2024-10-31	1.450
11	10293847H	2024-11-30	1.300
12	10293847H	2024-12-20	900
13	10293847H	2025-07-31	1.250
14	10293847H	2025-08-31	1.200
15	10293847H	2025-09-30	1.250
16	10293847H	2025-10-31	1.200
17	10293847H	2025-11-20	750
18	11223344N	2023-07-31	600
19	11223344N	2023-08-31	1.350
20	11223344N	2023-09-30	1.450
21	11223344N	2023-10-31	1.550
22	11223344N	2023-11-30	1.500
23	11223344N	2023-12-31	1.500
24	11223344N	2024-07-31	1.400

(170 Filas)

Y por último la tabla de ventas:
Me daba error al meter el .csv, todavía no entiendo el porque, la IA (Chat GPT), cogió mi .csv y lo convirtió en un SCRIPT SQL sin modificar nada y así me fue:

aceitunasdb | RECOGIDA | CAMPAÑA | COOPERATIVA | PAGO | *<AWS-AceitunasDB> Script-11 | VENTA

Propiedades | Datos | Diagrama

Enter a SQL expression to filter results (use Ctrl+Space)

	idVENTA	fechaEntrega	kilosEntregados	totalPagado	CAMPAÑA_idCAMPAÑA	COOPERATIVA_nombre	COOPERATIVA_direccion
1	1	2023-07-31	87.404	786.636	1	Aceitunas Morbe	Calle Lepanto, 8
2	2	2023-08-31	111.326	1.001.934	1	Aceitunas Morbe	Calle Lepanto, 8
3	3	2023-09-30	1.184.340	1.065.906	1	Aceitunas Morbe	Calle Lepanto, 8
4	4	2023-10-31	123.038	1.107.342	1	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
5	5	2023-11-30	118.501	106.650,9	1	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
6	6	2023-12-31	118.476	1.066.284	1	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
7	7	2024-07-31	201.400	181.260	2	Labradores de la Campiña	Bollullos de la Mitación
8	8	2024-08-31	201.394	1.812.546	2	Labradores de la Campiña	Bollullos de la Mitación
9	9	2024-09-30	192.634	173.370,6	2	Labradores de la Campiña	Bollullos de la Mitación
10	10	2024-10-31	201.014	180.912,6	2	Aceitunas Morbe	Calle Lepanto, 8
11	11	2024-11-30	201.019	180.917,1	2	Aceitunas Morbe	Calle Lepanto, 8
12	12	2024-12-20	128.637	115.773,3	2	Aceitunas Morbe	Calle Lepanto, 8
13	13	2025-07-31	133.020	119.718	3	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
14	14	2025-08-31	133.025	119.722,5	3	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
15	15	2025-09-30	129.010	116.109	3	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
16	16	2025-10-30	138.455	124.609,5	3	Labradores de la Campiña	Bollullos de la Mitación
17	17	2025-11-20	86.270	77.643	3	Labradores de la Campiña	Bollullos de la Mitación

Este es el SCRIPT:

```
INSERT INTO
VENTA(idVENTA,fechaEntrega,kilosEntregados,totalPagado,CAMPAÑA_idCAMPAÑA,COOPERATIVA_nombre,COOPERATIVA_direccion) VALUES
(1,'2023-07-31',87404.0,786636.0,1,'Aceitunas Morbe','Calle Lepanto, 8'),
(2,'2023-08-31',111326.0,1001934.0,1,'Aceitunas Morbe','Calle Lepanto, 8'),
(3,'2023-09-30',1184340.0,1065906.0,1,'Aceitunas Morbe','Calle Lepanto, 8'),
(4,'2023-10-31',123038.0,1107342.0,1,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(5,'2023-11-30',118501.0,106650.9,1,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(6,'2023-12-31',118476.0,1066284.0,1,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(7,'2024-07-31',201400.0,181260.0,2,'Labradores de la Campiña','Bollullos de la Mitación'),
(8,'2024-08-31',201394.0,1812546.0,2,'Labradores de la Campiña','Bollullos de la Mitación'),
(9,'2024-09-30',192634.0,173370.6,2,'Labradores de la Campiña','Bollullos de la Mitación'),
(10,'2024-10-31',201014.0,180912.6,2,'Aceitunas Morbe','Calle Lepanto, 8'),
(11,'2024-11-30',201019.0,180917.1,2,'Aceitunas Morbe','Calle Lepanto, 8'),
(12,'2024-12-20',128637.0,115773.3,2,'Aceitunas Morbe','Calle Lepanto, 8'),
(13,'2025-07-31',133020.0,119718.0,3,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(14,'2025-08-31',133025.0,119722.5,3,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(15,'2025-09-30',129010.0,116109.0,3,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(16,'2025-10-30',138455.0,124609.5,3,'Labradores de la Campiña','Bollullos de la Mitación'),
(17,'2025-11-20',86270.0,77643.0,3,'Labradores de la Campiña','Bollullos de la Mitación');
```

5. Consultas Multitabla SQL

Aquí van las consultas que nos piden:

5.1 Consulta 1 – (4 tablas)

Enunciado Consulta:
Mostrar cuántos días ha trabajado cada empleado en cada campaña.

SQL:

```sql
select e.nombre, c.año, COUNT(*) AS 'Días Trabajados'
from EMPLEADO e
    inner join RECOGIDOS rd
        on e.DNI = rd.EMPLEADO_DNI
    inner join RECOGIDA r
        on rd.RECOGIDA_fechaRecogida = r.fechaRecogida
    inner join `CAMPAÑA` c
        on r.CAMPAÑA_idCAMPAÑA = c.idCampaña
group by e.nombre, c.año
order by c.año, e.nombre;
```

Resultado:

```sql
-- Mostrar cuántos días ha trabajado cada empleado en cada campaña.
select e.nombre, c.año, COUNT(*) AS 'Días Trabajados'
from EMPLEADO e
    inner join RECOGIDOS rd
        on e.DNI = rd.EMPLEADO_DNI
    inner join RECOGIDA r
        on rd.RECOGIDA_fechaRecogida = r.fechaRecogida
    inner join `CAMPAÑA` c
        on r.CAMPAÑA_idCAMPAÑA = c.idCampaña
group by e.nombre, c.año
order by c.año, e.nombre;
```

EMPLEADO(+) 1

Enter a SQL expression to filter results (us

	nombre	año	Días Trabajados
1	Adrian	2023	158
2	Alberto	2023	160
3	Antonio	2023	158
4	Javier	2023	159
5	Lidia	2023	159
6	Lorena	2023	162
7	Pilar	2023	159
8	Raquel	2023	162
9	Rosa	2023	160
10	Victoria	2023	162
11	Adrian	2024	158
12	Alberto	2024	158
13	Antonio	2024	159

*"La consulta cuenta el número de días que cada empleado ha trabajado en cada campaña. Para ello se relacionan las tablas EMPLEADO, RECOGIDOS, RECOGIDA y CAMPAÑA mediante **inner join**."*

5.2 Consulta 2 – GROUP BY

Enunciado Consulta:

Mostrar los kilos totales recogidos por cada campaña (agrupado por año).

SQL:

```
select c.año, SUM(r.kilosRecogidos) AS 'Kilos Totales'
from CAMPAÑA c
    inner join RECOGIDA r
        on c.idCAMPAÑA = r.CAMPAÑA_idCAMPAÑA
group by c.año
order by c.año;
```

Resultado:

```
-- Mostrar los kilos totales recogidos por cada campaña (agrupado por año).
select c.año, SUM(r.kilosRecogidos) AS 'Kilos Totales'
from CAMPAÑA c
    inner join RECOGIDA r
        on c.idCAMPAÑA = r.CAMPAÑA_idCAMPAÑA
group by c.año
order by c.año;
```

AÑA 1

Enter a SQL expression to filter results (use Ctrl+Space)

año	kilosTotales
2023	695.273
2024	1.111.178
2025	619.895

"En esta consulta se utiliza la cláusula GROUP BY para agrupar las recogidas por campaña y calcular el total de kilos recogidos en cada una de ellas mediante la función SUM."

5.3 Consulta 3 – Subconsulta

Enunciado Consulta:

Empleados cuyo pago medio supera la media general, que se vea también su media de pago sin decimales.

SQL:

```
select e.nombre, concat(truncate(avg(p.importe), 0), '€') as 'Media de Pagos'
from EMPLEADO e
      inner join PAGO p
            on e.DNI = p.EMPLEADO_DNI
group by e.nombre
having AVG(p.importe) > (
      select AVG(importe)
      from PAGO
);
```

Resultado:

```
-- Empleados cuyo pago medio supera la media general, que se vea también su media de pago sin decimales.
select e.nombre, concat(truncate(avg(p.importe), 0), '€') as 'Media de Pagos'
from EMPLEADO e
    inner join PAGO p
        on e.DNI = p.EMPLEADO_DNI
group by e.nombre
having AVG(p.importe) > (
    select AVG(importe)
    from PAGO
);
```

PLEADO 1

Enter a SQL expression to filter results (use Ctrl+Space)

nombre	Media de Pagos
Javier	1261€
Antonio	1267€
Victoria	1273€
Alberto	1270€
Lidia	1258€

"Esta consulta muestra los empleados que, de media, han cobrado más que la media general de todos los pagos. Para ello se calcula la media de cada trabajador y se compara con la media total. También se eliminan los decimales y se añade el símbolo € para que el resultado sea más claro."

5.4 Consulta 4 – Funciones de fecha

Enunciado Consulta:

Duración de cada campaña en días.

SQL:

```
select c.año, CONCAT_WS(' / ',c.fechaInicio, c.fechaFin) as 'Rango de Fecha', DATEDIFF(c.fechaFin, c.fechaInicio) as 'Duración (días)'
from `CAMPAÑA` c
order by c.año;
```

Resultado:

```
-- Mostrar el año de la campaña, la fecha de inicio, la fecha de fin y los días que dura cada campaña.
select c.año, CONCAT_WS(' / ',c.fechaInicio, c.fechaFin) as 'Rango de Fecha', DATEDIFF(c.fechaFin, c.fechaInicio) as 'Duración (días)'
from `CAMPAÑA` c
order by c.año;
```

PAÑA 1

Enter a SQL expression to filter results (use Ctrl+Space)

año	Rango de Fecha	Duración (días)
2023	2023-07-10 / 2023-12-31	174
2024	2024-07-01 / 2024-12-20	172
2025	2025-07-01 / 2025-11-20	142

"La consulta muestra la duración de cada campaña en días. Para ello se utiliza la función DATEDIFF, que calcula la diferencia en días entre la fecha de fin y la fecha de inicio."

5.5 Consulta 5 – Funciones de texto

Enunciado Consulta:

Mostrar el nombre de las cooperativas en mayúsculas, los apellidos en minúsculas, el nombre completo, la longitud del nombre, las tres primeras letras del DNI y los dos últimos dígitos del teléfono

SQL:

```
select
        UPPER(e.nombre) as 'Nombre Mayúsculas',
        LOWER(e.apellidos) as 'Apellidos Minúsculas',
        CONCAT_WS(' ',e.nombre, e.apellidos) as 'Nombre Completo',
        LENGTH(e.nombre) as 'Longitud Nombre',
        LEFT(e.DNI, 3) as 'Inicio DNI',
        RIGHT(e.telefono, 2) as 'Últimos 2 Dígitos del Teléfono'
from EMPLEADO e
order by e.nombre;
```

Resultado:

```
-- Mostrar el nombre en mayúsculas, los apellidos en minúsculas,
-- el nombre completo, la longitud del nombre, las 3 primeras letras del DNI (Empezando por la izquierda) y el los 2 últimos dígitos del telefono.
select
    UPPER(e.nombre) as 'Nombre Mayúsculas',
    LOWER(e.apellidos) as 'Apellidos Minúsculas',
    CONCAT_WS(' ',e.nombre, e.apellidos) as 'Nombre Completo',
    LENGTH(e.nombre) as 'Longitud Nombre',
    LEFT(e.DNI, 3) as 'Inicio DNI',
    RIGHT(e.telefono, 2) as 'Últimos 2 Dígitos del Teléfono'
from EMPLEADO e
order by e.nombre;
```

Resultados 1

Enter a SQL expression to filter results (use Ctrl+Space)

	Nombre Mayúsculas	Apellidos Minúsculas	Nombre Completo	Longitud Nombre	Inicio DNI	Últimos 2 Dígitos del Teléfono
1	ADRIAN	nuñez cano	Adrian Nuñez Cano	6	241	88
2	ALBERTO	vazquez ruiz	Alberto Vazquez Ruiz	7	192	22
3	ANTONIO	sanchez martinez	Antonio Sanchez Martinez	7	129	44
4	JAVIER	diaz moreno	Javier Diaz Moreno	6	102	99
5	LIDIA	rojas blanco	Lidia Rojas Blanco	5	223	77
6	LORENA	ibañez peña	Lorena Ibañez Peña	6	129	55
7	MOHAMED	amead el haouzi	Mohamed Amead El Haouzi	7	549	66
8	PILAR	gutierrez blanco	Pilar Gutierrez Blanco	5	112	55
9	RAQUEL	ramos castillo	Raquel Ramos Castillo	6	135	00
10	ROSA	navarro cano	Rosa Navarro Cano	4	223	33
11	VICTORIA	iglesias soto	Victoria Iglesias Soto	8	132	77

“La consulta utiliza varias funciones de texto sobre la tabla EMPLEADO. Se muestran los nombres en mayúsculas, los apellidos en minúsculas, el nombre completo, la longitud del nombre, las tres primeras letras del DNI y los dos últimos dígitos del teléfono.”

5.6 Consulta 6 – ISNULL / IFNULL

Enunciado Consulta:

Mostrar los manijeros y cuántos días han trabajado, si no han trabajado ningún día, que aparezca 0.

SQL:

```
select  e.nombre, m.zonaResponsable,
IFNULL(COUNT(rd.RECOGIDA_fechaRecogida), 0) as 'Días Trabajados'
from MANIJERO m
      inner join EMPLEADO e
            on e.DNI = m.EMPLEADO_DNI
      left join RECOGIDOS rd
            on rd.EMPLEADO_DNI = e.DNI
group by e.nombre, m.zonaResponsable
order by `Días Trabajados` desc, e.nombre;
```

Resultado:

```
-- Mostrar los manijeros y cuántos días han trabajado, si no han trabajado ningún día, que aparezca 0.
select  e.nombre, m.zonaResponsable, IFNULL(COUNT(rd.RECOGIDA_fechaRecogida), 0) as 'Días Trabajados'
from MANIJERO m
    inner join EMPLEADO e
        on e.DNI = m.EMPLEADO_DNI
    left join RECOGIDOS rd
        on rd.EMPLEADO_DNI = e.DNI
group by e.nombre, m.zonaResponsable
order by `Días Trabajados` desc, e.nombre;
```

PLEADO(+) 1

Enter a SQL expression to filter results (use Ctrl+Space)

nombre	zonaResponsable	Días Trabajados
Javier	Finca El Pintao - Sector Norte	429
Pilar	Finca El Pintao - Sector Sur	419
Mohamed	Finca El Pintao - Sector Medio	0

"La consulta muestra los manijeros y los días que han trabajado. Se usa LEFT JOIN para que también aparezcan los que no han trabajado ningún día, y IFNULL para mostrar 0 en esos casos."

5.7 Consulta 7 – Consulta compleja

Enunciado Consulta:

El empleado que ha trabajado más días que todos los empleados sin usar limit.

SQL:

```
select e.nombre, COUNT(*) as 'Días Trabajados'
from EMPLEADO e
        inner join RECOGIDOS r
                on r.EMPLEADO_DNI = e.DNI
group by e.DNI
having COUNT(*) >= ALL (
        select COUNT(*)
        from EMPLEADO e2
                inner join RECOGIDOS r2
                on r2.EMPLEADO_DNI = e2.DNI
                group by e2.DNI
)
order by e.nombre;
```

Resultado:

```
-- El empleado que ha trabajado más días que todos los empleados sin usar limit.
select e.nombre, COUNT(*) as 'Días Trabajados'
from EMPLEADO e
    inner join RECOGIDOS r
        on r.EMPLEADO_DNI = e.DNI
group by e.DNI
having COUNT(*) >= ALL (
    select COUNT(*)
    from EMPLEADO e2
        inner join RECOGIDOS r2
        on r2.EMPLEADO_DNI = e2.DNI
        group by e2.DNI
)
order by e.nombre;
```

MPLEADO 1

Enter a SQL expression to filter results (use Ctrl+Space)

nombre	Días Trabajados
Victoria	433

"La consulta obtiene el empleado que más días ha trabajado. Se calcula el total de días por empleado y se compara con todos los totales usando ALL, sin usar LIMIT."

6. Vistas

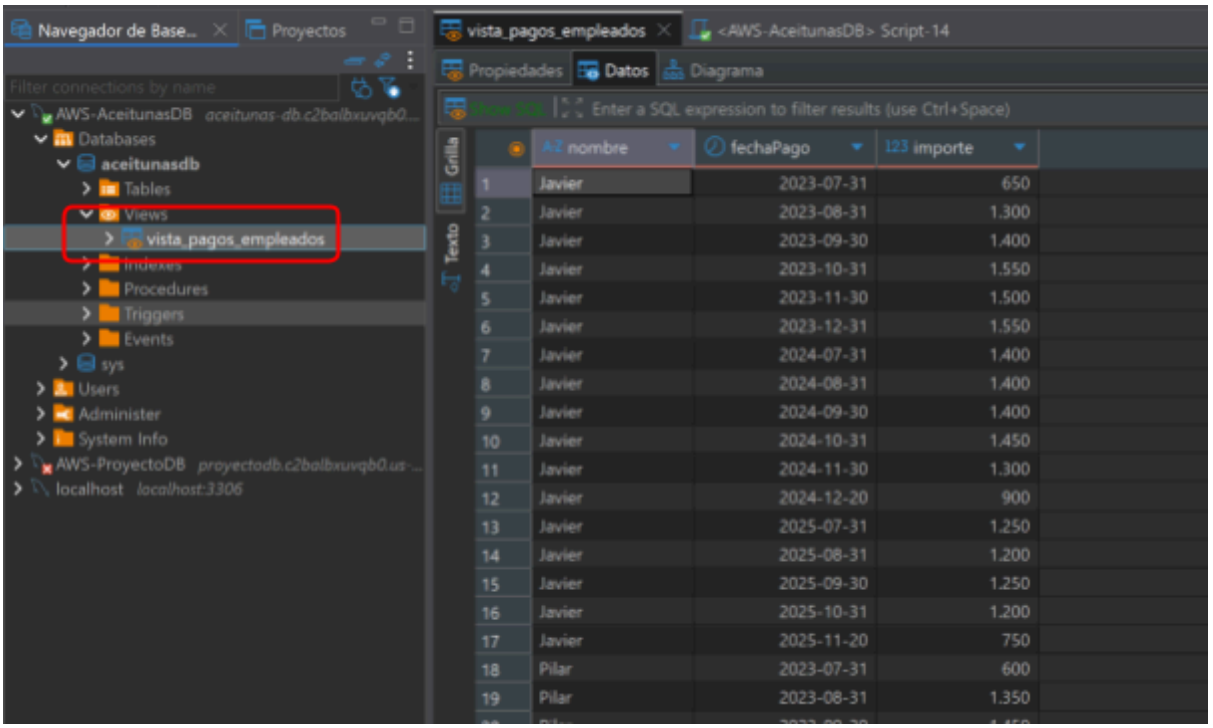
6.1 Vista 1

Enunciado de la vista:
Crear una vista que muestre el nombre del empleado, la fecha del pago y el importe.

SQL:

```
create view vista_pagos_empleados as
select e.nombre, p.fechaPago, p.importe
from EMPLEADO e
        inner join PAGO p
                on e.DNI = p.EMPLEADO_DNI;
```

Resultado:



"Se crea una vista para consultar de forma rápida los pagos realizados a cada empleado, mostrando su nombre, la fecha del pago y el importe."

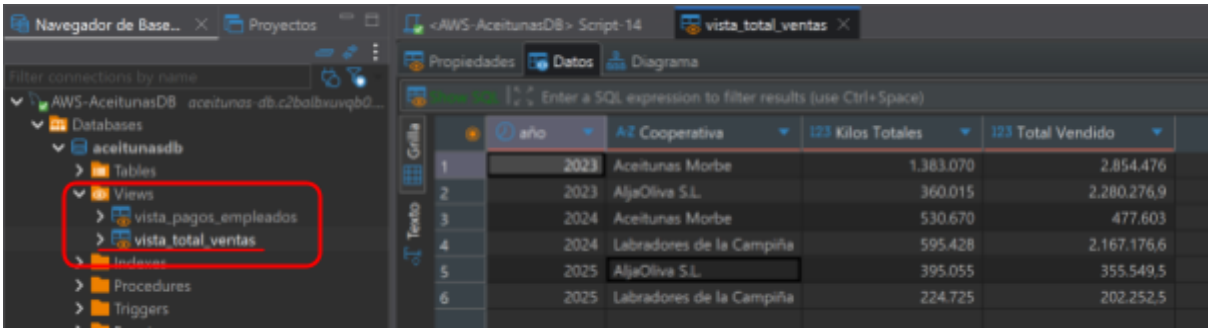
6.2 Vista 2

Enunciado de la vista:
Crear una vista que muestre el total de kilos y dinero vendido por campaña y cooperativa, sin decimales y con símbolo €.

SQL:

```
create view vista_total_ventas as
select
        c.año,
        v.COOPERATIVA_nombre as 'Cooperativa',
        SUM(v.kilosEntregados) as 'Kilos Totales',
        SUM(v.totalPagado) as 'Total Vendido'
from CAMPAÑA c
        inner join VENTA v
                on c.idCampaña =
v.CAMPAÑA_idCAMPAÑA
group by c.año, v.COOPERATIVA_nombre
order by c.año, v.COOPERATIVA_nombre;
```

Resultado:



"Esta vista resume las ventas agrupándolas por campaña y cooperativa, mostrando el total de kilos entregados y el total vendido, facilitando la consulta de resultados por temporada."

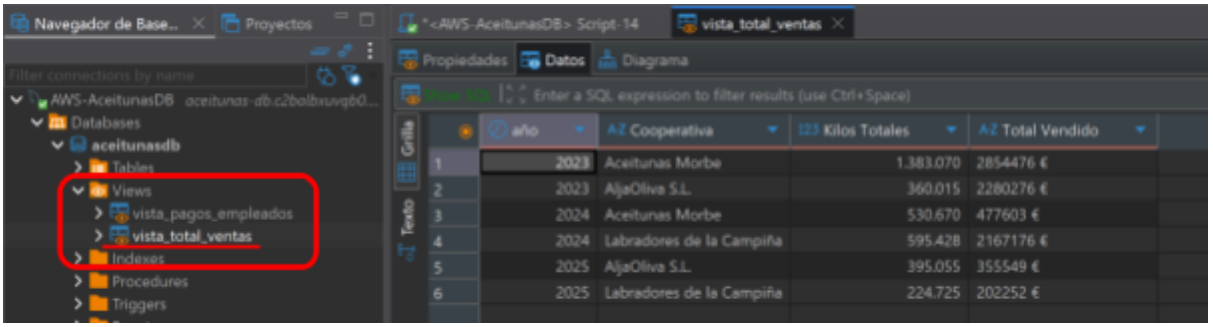
Lo hice así me dí cuenta de que salían decimales así que con ddl, vamos a borrar esta lista, y crearemos una nueva cone ste SQL:

```
create view vista_total_ventas as
select
        c.año,
        v.COOPERATIVA_nombre as 'Cooperativa',
        TRUNCATE(SUM(v.kilosEntregados),0) as 'Kilos Totales',
        CONCAT(TRUNCATE(SUM(v.totalPagado),0), ' €') as 'Total Vendido'
from CAMPAÑA c
        inner join VENTA v
                on c.idCampaña = v.CAMPAÑA_idCAMPAÑA
group by c.año, v.COOPERATIVA_nombre
order by c.año, v.COOPERATIVA_nombre;
```

Así que ponemos el ddl:

```
DROP VIEW aceitunasdb.vista_total_ventas;
```

Y ahora si ponemos el script y quedaría así:



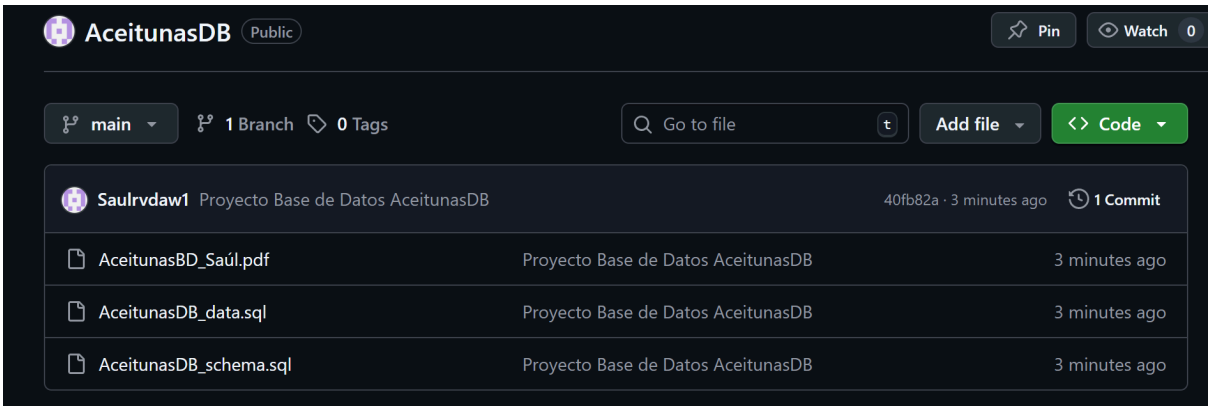
7. Scripts SQL del Proyecto

En este proyecto se han usado distintos scripts SQL para trabajar con la base de datos.
Estos scripts se han utilizado para hacer las consultas y para crear las vistas que se muestran en los apartados anteriores.

Todos los scripts se han ejecutado desde DBeaver sobre la base de datos alojada en AWS, por lo que no ha sido necesario incluirlos de nuevo en este apartado.
Por ejemplo de carga masiva → 4. Carga Masiva de Datos
o también de consultas → 5. Consultas Multitabla SQL
O de vistas → 6. Vistas

8. Archivos en git

Aquí podemos ver los 3 archivos subidos correctamente a GitHub:



En el repositorio se encuentran el script de creación de la base de datos (`AceitunasDB_schema.sql`), el script de carga de datos (`AceitunasDB_data.sql`) y el documento PDF del proyecto.

Enlace al repositorio GitHub:

https://github.com/Saulrvdaw1/AceitunasDB

9. Funciones y Procedimientos

1° Función

Descripción de la función → La función calcula el pago mensual de un empleado contando los días trabajados en un mes y multiplicándolos por 50€.

Código de Función

```
delimiter //
create function calcular_pago_mensual(p_dni
varchar(9), p_año int, p_mes int)
returns double
deterministic
begin
    declare total double;
    select (count(*) * 50) into total
    from RECOGIDOS
    where empleado_dni = p_dni
     and year(recogida_fecharecogida) = p_año
     and month(recogida_fecharecogida) = p_mes;
    return total;
end //
delimiter ;
```

Comprobamos su funcionamiento:

```
select calcular_pago_mensual('10293847H', 2025, 7);
```

ultados 1

Enter a SQL expression to filter results (use Ctrl+Space)

123 calcular_pago_mensual('10293847H', 2025, 7)
1.250

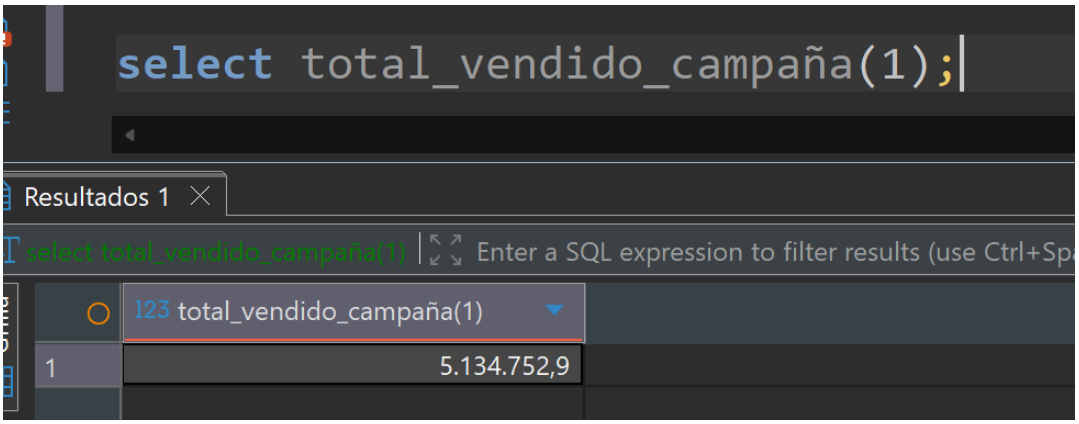
2º Función

Descripción de la función → suma todo el dinero vendido en una campaña.

Código de Función

```
delimiter //
create function total_vendido_campaña(p_idcampaña
int)
returns double
deterministic
begin
    declare total double;
    select sum(totalpagado) into total
    from VENTA
    where campaña_idcampaña = p_idcampaña;
    return total;
end //
delimiter ;
```

Comprobamos su funcionamiento:



1º Procedimiento

Descripción del Procedimiento → mostrar el pago mensual de un empleado, este procedimiento usa la **función 1**, así cumples lo de que uno use una función.

Código del Procedimiento

```
delimiter //
create procedure mostrar_pago_mensual(in p_dni varchar(9), p_año int, p_mes int)
begin
    select p_dni as dni, p_año as año, p_mes as mes,
           calcular_pago_mensual(p_dni, p_año, p_mes) as pago_mensual;
end //
delimiter ;
```

Comprobamos su funcionamiento:

```
call mostrar_pago_mensual('10293847H', 2025, 7);
```

Resultados 1 | Estadísticas 1

call mostrar_pago_mensual('10293847H', 2025, 7) | Enter a SQL expression to filter results (use Ctrl+Space)

	dni	año	mes	pago_mensual
1	10293847H	2.025	7	1.250

2º Procedimiento

Descripción del Procedimiento → insertar el pago mensual en la tabla pago, este también usa la función 1 y mete el pago en la tabla.

Código del Procedimiento

```
delimiter //
create procedure insertar_pago_mensual(in p_dni varchar(9), in p_año int, in p_mes int)
begin
    insert into PAGO(empleado_dni, fechapago, importe)
    values ( p_dni, concat(p_año, '-', p_mes, '-01'), calcular_pago_mensual(p_dni, p_año,
p_mes));
end //
delimiter ;
```

Comprobamos que funciona ejecutando el procedimiento con un `CALL` y revisando la tabla de pagos:

```
call insertar_pago_mensual('10293847H', 2025, 7);
```

dísticas 1

	Value
Rows	1

Como el empleado ya tiene muchos pagos registrados, no se aprecia bien el nuevo. Por eso se hace una prueba con un año en el que no ha trabajado (2026), para que se genere un pago de 0€ y así poder comprobar claramente que el procedimiento funciona bien y saldría así:

11	10293847H	2024-11-30	1.300
12	10293847H	2024-12-20	900
13	10293847H	2025-07-01	1.250
14	10293847H	2025-07-31	1.250
15	10293847H	2025-08-31	1.200
16	10293847H	2025-09-30	1.250
17	10293847H	2025-10-31	1.200
18	10293847H	2025-11-20	750
19	10293847H	2026-07-01	0
20	11223344N	2023-07-31	600
21	11223344N	2023-08-31	1.350
22	11223344N	2023-09-30	1.450

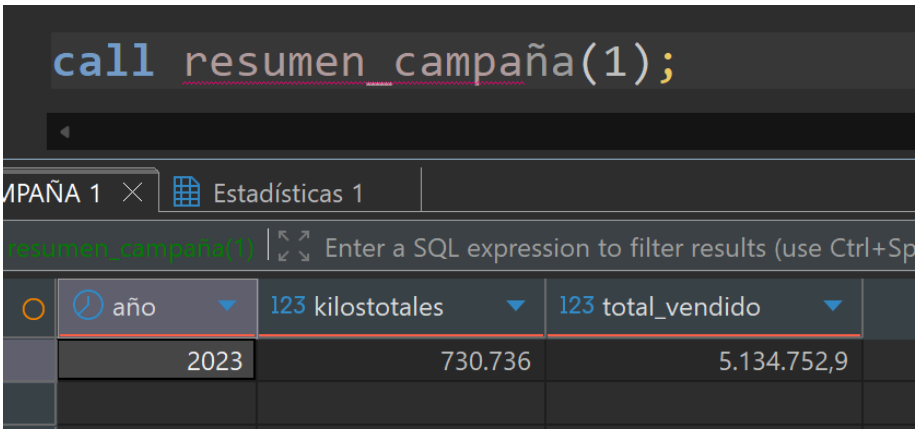
3° Procedimiento

Descripción del Procedimiento → mostrar resumen de una campaña muestra kilos totales y dinero vendido.

Código del Procedimiento

```
delimiter //
create procedure resumen_campaña(in p_idcampaña int)
begin
    select c.año, c.kilostotales, total_vendido_campaña(p_idcampaña) as total_vendido
    from CAMPAÑA c
    where c.idcampaña = p_idcampaña;
end //
delimiter ;
```

Comprobamos su funcionamiento:



Mini Resumen de lo hecho:

> Se han creado dos funciones y tres procedimientos para automatizar tareas dentro de la base de datos, una de las funciones calcula el pago mensual de un empleado según los días trabajados, y otra obtiene el total vendido en una campaña, además, dos (1 y 2) procedimientos utiliza una de estas funciones para mostrar e insertar pagos mensuales.

10. Triggers

1° Trigger

Descripción del Trigger → impedir pagos negativos o 0, si intentas meter un pago con importe menor o igual que 0, tiene que dar error.

Código del Trigger

```
delimiter //
create trigger comprobar_importe_pago
before insert on PAGO
for each row
begin
    if new.importe <= 0 then
        signal sqlstate '45000'
        set message_text = 'El importe del pago no puede ser menor o igual que 0';
    end if;
end //
delimiter ;
```

Comprobamos su funcionamiento intentando insertar un pago con 0€:

```
insert into PAGO (empleado_dni, fechapago, importe)
values ('10293847H', '2025-11-30', 0);
```

Resultados 1

insert into PAGO (empleado_dni, fechapago, importe) values ('1029384 | Enter a SQL expression to filter results (use Ctrl+Space)

```
SQL Error [1644] [45000]: El importe del pago no puede ser menor o igual que 0
```

2º Trigger

Descripción del Trigger → actualizar kilos totales de campaña al insertar una recogida, cuando insertas una nueva recogida, suma esos kilos en la campaña correspondiente.

<u>Código del Trigger</u>

```
delimiter //
create trigger actualizar_kilos_campaña
after insert on RECOGIDA
for each row
begin
    update CAMPAÑA
    set kilostotales = kilostotales + new.kilosrecogidos
    where idcampaña = new.campaña_idcampaña;
end //
delimiter ;
```

Comprobamos su funcionamiento
Primero miramos antes:

```
select idcampaña, año, kilostotales
from CAMPAÑA
where idcampaña = 3;
```

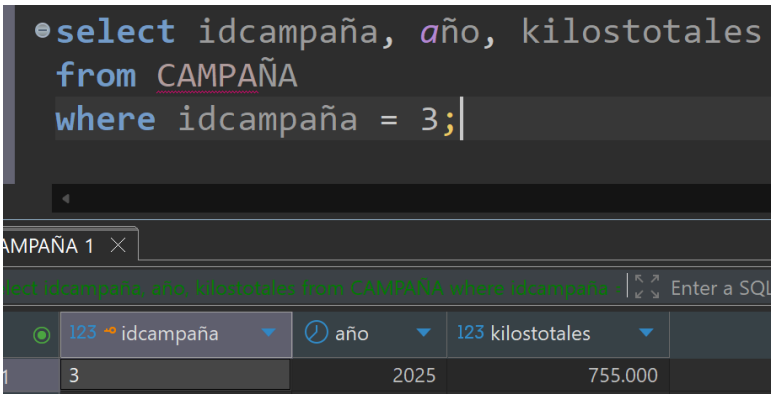
MPAÑA 1 | Enter a SQL

123 idcampaña	año	123 kilostotales
3	2025	750.000

Insertamos un dato:

```
insert into RECOGIDA (fecharecogida, kilosrecogidos, campaña_idcampaña)
values ('2025-11-21', 5000, 3);
```

Y volvemos a mirar:



Y vemos que suben los 5.000Kg correctamente.

Mini Resumen de lo hecho:

> Se han creado dos triggers, uno impide insertar pagos con importe menor o igual que 0, y otro actualiza automáticamente los kilos totales de una campaña al insertar una nueva recogida, en los 2 se ha se ha hecho una comprobación para ver cómo salta el trigger.

11. Conclusión y valoración personal

Este proyecto me ha ayudado a entender mejor cómo funcionan las bases de datos en la práctica. No solo he diseñado la base de datos y las tablas, sino que también he trabajado con datos reales, cargándola mediante archivos CSV y scripts SQL, lo que me ha servido para ver cómo se manejan grandes cantidades de información.

Además, he realizado distintas consultas SQL utilizando INNER JOIN, GROUP BY, subconsultas y funciones básicas, lo que me ha permitido sacar información útil de la base de datos y comprobar que los datos estaban bien relacionados. También he creado vistas para facilitar la consulta de los pagos y de las ventas por campaña.

Por otra parte, he trabajado con funciones, procedimientos y triggers, lo que me ha ayudado a entender cómo automatizar tareas dentro de la base de datos. Por ejemplo, he creado funciones para calcular pagos y procedimientos para insertar datos automáticamente, además de triggers que controlan o actualizan información sin necesidad de hacerlo manualmente.

También he aprendido a usar Git y GitHub para guardar el proyecto y llevar un control de versiones, lo que es muy útil para no perder el trabajo y poder subir cambios fácilmente.

En general, ha sido un proyecto bastante completo porque me ha permitido aplicar lo aprendido en clase a un caso real, como la gestión de la recogida de aceitunas, y me ha ayudado a reforzar conceptos que me servirán en segundo de DAW y en proyectos más grandes.